

1-O que e um Tipo Abstrato de Dados (TAD) e qual a característica fundamental na sua utilização?

Um Tipo Abstrato de Dados (TAD) é um modelo de estrutura de dados que define quais operações podem ser realizadas sobre os dados, sem mostrar como essas operações são implementadas internamente.

Ou seja, o TAD descreve o comportamento do tipo de dado, especificando:

- quais dados ele armazena;
- quais operações podem ser feitas com ele (inserir, remover, consultar, etc.).

2. Quais as vantagens de se programar com TADs?

- Organização do código
O programa fica mais estruturado, pois os dados e suas operações ficam agrupados em um único tipo.
- Reutilização de código
Um TAD pode ser utilizado em diferentes programas sem precisar ser reescrito.
- Facilidade de manutenção
Se for necessário alterar a implementação interna, isso pode ser feito sem afetar o restante do programa.

3. Crie um TAD que implemente o controle de uma conta bancaria, implemente funções para inicializar, deposito, sacar e imprimir o saldo.

```
#include<stdio.h>

#include<stdlib.h>

#include<strings.h>

typedef struct
{
    char nomeCliente[60];

    float saldo;
```

```
} Conta;

Conta inicializar (Conta c){

    c.saldo = 0;

    return c;

}

Conta depositar(Conta c, float valor){

    c.saldo += valor;

    return c;

}

Conta sacar(Conta c, float valor){

    if(valor > c.saldo){

        printf("Saldo indisponivel!");

        return c;

    }

    else{

        c.saldo -= valor;

        return c;

    }

}

void imprimir(Conta c){

    printf("%s \nSaldo: R$%.2f\n",c.nomeCliente, c.saldo);

}
```

```

}

int main() {

    Conta cliente1;

    printf("Digite seu nome: ");

    scanf("%s", cliente1.nomeCliente);

    cliente1 = inicializar(cliente1);

    cliente1 = depositar(cliente1, 150.00);

    cliente1 = sacar(cliente1, 55.50);

    imprimir(cliente1);

    return 0;

}

```

4. Utilizando a alocação dinamica de memoria crie uma função para criar uma nova conta bancaria:

```

#include<stdio.h>

#include<stdlib.h>

#include<strings.h>

```

```
typedef struct
{
    char nomeCliente[60];

    float saldo;
} Conta;

Conta* criarConta(){

    Conta *c = malloc(sizeof(Conta));

    c->saldo = 0;

    return c;
}

int main(){

    Conta *cliente1;

    cliente1 = criarConta();

    printf("Digite o nome do cliente: ");

    scanf("%s", cliente1->nomeCliente);

    cliente1->saldo = 200;

    printf("Cliente: %s\n", cliente1->nomeCliente);
```

```
printf("Saldo: R$%.2f\n", cliente1->saldo);

free(cliente1);

return 0;
}
```